

Document number: P2634R0
Date: 2022-08-15
Reply-to: Justin Cooke <jgc@cems.de>
Target audience: EWG

Allow qualifiers in constructor declarations

Proposal: To allow *cv-qualifiers* and *ref-qualifiers* in constructor declarations.

Motivation: Currently, the same constructors are called for `const` and non-`const` class objects, and for named and temporary objects. It is often useful to distinguish between these cases: for example, to prohibit the construction of temporary objects of a given type, or to provide a slimmed-down constructor suitable only for `const` objects. Qualifiers in constructor declarations would enable such distinctions.

Description: The semantics of a `const` or `volatile` qualifier differ between constructors and ordinary member functions. A `const` ordinary member function can be called on both `const` and non-`const` objects, while a non-`const` ordinary member function can only be called on non-`const` objects. By contrast, a `const` constructor can only be used to construct a `const` object, while a non-`const` constructor can be used to construct either a `const` or a non-`const` object. Likewise for `volatile`. An `&`-qualified constructor can only be invoked to construct an lvalue. An `&&`-qualified constructor can only be invoked to construct an rvalue.

Effect on existing code: None. Code using the proposed feature is ill-formed under C++20 and earlier standards.

Wording: (edits to N4910)

11.4.5 Constructors

[class.ctor]

11.4.5.1 General

[class.ctor.general]

¹ A declarator declares a constructor if it is a function declarator (9.3.4.6) of the form *ptr-declarator* (*parameter-declaration-clause*)) *cv-qualifier-seq*_{opt} *ref-qualifier*_{opt} *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt}

...

⁵ A constructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics (9.2.9.2) are not applied on an object under construction. They come into effect when the constructor for the most derived object (6.7.2) ends. A `const`-qualified constructor can only be invoked for a `const` object. A `volatile`-qualified constructor can only be invoked for a `volatile` object.

^{5A} A constructor whose *ref-qualifier* is `&` can only be invoked to construct an lvalue. A constructor whose *ref-qualifier* is `&&` can only be invoked to construct an rvalue.

[Example 3]

```
struct S {  
    S() &; // #1  
    S() &&; // #2  
    S() const &; // #3  
};  
S x; // calls #1  
S(); // calls #2  
const S y; // calls #3  
new const S; // calls #3
```

— *end example*].

12.2.3 Viable functions

[over.match.viable]

...

³ Second, for a function to be viable, if it has associated constraints (13.5.3), those constraints shall be satisfied (13.5.2).

^{3.1} A cv-qualified or ref-qualified constructor shall satisfy the restrictions specified in 11.4.5.1.

Note: While `const&&` would be a permissible qualifier on a constructor, an explicit cast would be needed to invoke such a constructor and usage would likely be rare.